

A9. Reverse an Array (6 pts)

Reversing an array is not a difficult task, but handling dynamic memory allocation properly is crucial in C programming.

In this task, you will implement a function that performs this reversal. The judging system will automatically compile your code together with the provided main program, so your submission should **only contain the function implementation**.

➤ Your Task:

1. Download the files  a9.c,  a9.h, and  main.c from the site.
2. Implement the function `reverse_array()` as described.
3. Submit only your modified  a9.c file.

Please make sure the directory structure looks like this before compiling on your computer:

```
.
├── a9.c
├── a9.h
└── main.c
```

You can compile the code using the command:

```
gcc -std=c2x -o a9.exe main.c a9.c -lm
```

Since you can only upload a9.c to the online judge, modifying main.c and a9.h is **not allowed**.

🔗 What the program does:

You are required to implement the following function defined in a9.h:

```
void reverse_array(const int n);
```

which takes a length n , allocates the array with size n , reads the array from standard input and prints the reversed array.

⚠ Constraints:

- The number of testcases $1 \leq t \leq 5$
- The array length $1 \leq n \leq 2 \times 10^6$
- Each element value $-10^{12} \leq \text{val} \leq 10^{12}$
- Take your own risk to use global arrays.
- Take care of the memory limit!
- You may define helper functions inside a9.c if needed.

📖 Input Format:

- The first line contains one integers t , the number of testcases.
- For each testcase:
 - ▶ The first line contains two integer n , the length of the array.
 - ▶ The second line contains n integers, the elements of the array separated by spaces.

Output Format:

- There are t testcases. For each testcase:
 - The output should be a single line with the reversed array elements separated by spaces.

Sample Test Cases:

Sample Input 1:

```
4
4
1 2 3 4
5
5 4 3 2 1
3
10 40 39
8
920 280 300 560 500 600 740 800
```

Sample Output 1:

```
OK
4 3 2 1
1 2 3 4 5
39 40 10
800 740 600 500 560 300 280 920
```

Sample Input 2:

```
1
100
990032025 555072665 655366972 626...
```

(Check all-tasks.zip for the complete file.)

Sample Output 2:

```
OK
345974908 690798808 957890290 144...
```

(Check all-tasks.zip for the complete file.)